SENTIMENT ANALYSIS OF PRODUCT REVIEWS

Complete Project Document

Project question: How well can text analytics identify positive, neutral and negative opinions in product reviews?

Methods: Natural Language Processing (NLP), TF-IDF, Multinomial Naive Bayes, Logistic Regression, class-wise evaluation and visualisation.

Tools: Python, pandas, scikit-learn, matplotlib and Jupyter/Google Colab.

This document combines the project report, dataset card, notebook workflow, and review analyser code in one PDF.

Data integrity note: The included 30-review sample is illustrative only. Use a documented public dataset before reporting final performance scores.

Project report

Abstract

This project investigates whether text analytics can classify product reviews into positive, neutral and negative sentiment. The workflow includes data checks, rating-to-label mapping, stratified train/test split, TF-IDF features, two baseline classifiers, class-wise metrics, plots and error analysis.

Introduction

Online product reviews contain opinions about quality, value, delivery, usability and customer support. NLP can help summarise large collections of reviews, but model predictions are estimates and can miss context.

Objectives

Inspect and document a public dataset; audit missing values, duplicates, language, length and class balance; compare two models; evaluate precision, recall and F1 per class; visualise common terms; inspect errors; and build a simple review analyser.

Dataset and labels

Choose a public product-review dataset with text and ratings or sentiment labels. Record exact title, version, URL, publisher, licence, date accessed, permitted use and restrictions. If using 1–5 star ratings, map 1–2 to negative, 3 to neutral, and 4–5 to positive. This is an assumption, not perfect ground truth; prefer reliable human-annotated sentiment labels when available.

Methodology

Clean rows with missing or empty text, invalid ratings and exact duplicates. Inspect review length and class distribution. Use a stratified split and fixed random seed. Fit TF-IDF on training data only, then train Multinomial Naive Bayes and Logistic Regression. Evaluate on the held-out test set.

Evaluation

Precision measures how reliable a class prediction is; recall measures how many true examples of a class are found; F1 balances precision and recall; macro-F1 averages the three class F1 scores equally. Also inspect a confusion matrix. Accuracy alone can hide weak performance on a minority class.

Visualisation and error analysis

The notebook creates class distribution and review-length plots, a confusion matrix, top TF-IDF terms by class, and a table of prediction errors. Review errors for sarcasm, negation, mixed opinions, short reviews, ambiguous wording and rating-text disagreement.

Prototype

The Python script accepts a review and returns a predicted sentiment, class probabilities and a cautionary explanation. Predictions should not be treated as facts or used as the sole basis for consequential decisions.

Limitations

Rating-derived labels may not match text sentiment. Bag-of-words models struggle with sarcasm, negation, mixed opinions and domain-specific language. Performance may change across products, languages, regions and time periods. Follow the dataset licence and protect personal information.

Conclusion

TF-IDF with conventional classifiers is an understandable baseline for three-class sentiment analysis. Real performance must be established by running the notebook on the selected public dataset and reviewing class-wise metrics and errors. Do not present demo-set scores as real-world evidence.

Dataset card and reproducibility checklist

• Exact dataset title and version: complete before submission.

• Dataset landing page/download URL and publisher/authors: record the exact source.

• Licence and permitted uses: verify current terms before reuse or redistribution.

• Date accessed and fields used: record in the dataset card.

• Cleaning counts: rows before and after removing missing values and duplicates.

• Keep the fixed random seed, preprocessing choices, package versions, model settings, metrics and plots.

Notebook workflow and code appendix

Sentiment Analysis of Product Reviews

Project question: How well can text analytics identify positive, neutral and negative opinions in product reviews?

This notebook follows a reproducible workflow: dataset audit → transparent rating mapping → stratified train/test split → TF-IDF features → two baseline classifiers → per-class precision/recall/F1 → visualisations → error analysis.

> **Data integrity:** `demo_reviews.csv` is a tiny illustrative dataset made for testing the code. It is not a public benchmark and its scores must not be presented as real-world performance. For the final experiment, place a permitted public dataset in `reviews.csv` with `reviewText` and `rating` columns and record its source/licence in `dataset_card.md`.

Python code cell

```
import re, os, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, ConfusionMatrixDisplay, accuracy_score

RANDOM_STATE = 42
DATA_PATH = "reviews.csv" if os.path.exists("reviews.csv") else "demo_reviews.csv"
df = pd.read_csv(DATA_PATH)
print("Loaded:", DATA_PATH, "| rows:", len(df))
df.head()
```

1. Dataset audit and sentiment labels

The notebook expects `reviewText` and `rating`. For a 1–5 rating scale, it maps 1–2 to negative, 3 to neutral and 4–5 to positive. This is an assumption, not a perfect substitute for text labels. If the chosen dataset has explicit sentiment labels, adapt this cell and document the label definitions.

Python code cell

```
required = {"reviewText", "rating"}
missing = required - set(df.columns)
if missing:
    raise ValueError(f"Missing required columns: {missing}. Rename your columns to reviewText and
    rating.")
df = df.dropna(subset=["reviewText", "rating"]).copy()
df["reviewText"] = df["reviewText"].astype(str).str.strip()
df = df[df["reviewText"].str.len() > 0]
df["rating"] = pd.to_numeric(df["rating"], errors="coerce")
df = df.dropna(subset=["rating"])
df = df.drop_duplicates(subset=["reviewText"])
df = df[df["rating"].between(1, 5)]
df["sentiment"] = np.select(
    [df["rating"] <= 2, df["rating"] == 3, df["rating"] >= 4],
    ["negative", "neutral", "positive"], default=None
)
print("Rows after cleaning:", len(df))
print("\nClass counts:")
print(df["sentiment"].value_counts())
print("\nMissing values:")
print(df[["reviewText", "rating", "sentiment"]].isna().sum())
df["review_length"] = df["reviewText"].str.split().str.len()
print("\nReview length summary:")
```

```
display(df["review_length"].describe())
```

Python code cell

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
df["sentiment"].value_counts().reindex(["negative","neutral","positive"]).plot(kind="bar", ax=axes[0])
axes[0].set_title("Sentiment class distribution")
axes[0].set_xlabel("Sentiment"); axes[0].set_ylabel("Review count")
axes[1].hist(df["review_length"], bins=20)
axes[1].set_title("Review length distribution")
axes[1].set_xlabel("Words per review"); axes[1].set_ylabel("Frequency")
plt.tight_layout()
plt.show()
```

2. Stratified train/test split

The held-out test set is used only for final evaluation. A stratified split attempts to preserve class proportions. If any class has too few examples, collect more data before evaluating; do not silently remove a class.

Python code cell

```
X = df["reviewText"].astype(str)
y = df["sentiment"].astype(str)
counts = y.value_counts()
if len(counts) < 3 or counts.min() < 2:
    raise ValueError("Need at least 2 examples in each of the three classes. Use a larger dataset.")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y
)
print(f"Train: {len(X_train)} | Test: {len(X_test)}")
```

3. TF-IDF baseline and two classifiers

TF-IDF weights terms by their importance within a review relative to the corpus. The two models are Multinomial Naive Bayes and Logistic Regression. The vectorizer is fitted inside each pipeline using training data only, preventing test-set vocabulary leakage.

Python code cell

```
models = {
    "Multinomial Naive Bayes": Pipeline([
        ("tfidf", TfidfVectorizer(lowercase=True, strip_accents="unicode",
                                  ngram_range=(1,2), min_df=1, max_features=50000)),
        ("clf", MultinomialNB())
    ]),
    "Logistic Regression": Pipeline([
        ("tfidf", TfidfVectorizer(lowercase=True, strip_accents="unicode",
                                  ngram_range=(1,2), min_df=1, max_features=50000)),
        ("clf", LogisticRegression(max_iter=1000, class_weight="balanced",
                                   random_state=RANDOM_STATE))
    ])
}
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    results[name] = {
        "model": model,
        "pred": pred,
        "accuracy": accuracy_score(y_test, pred),
        "report": classification_report(y_test, pred, labels=["negative","neutral","positive"],
                                        output_dict=True, zero_division=0)
    }
    print("\n###", name)
    print(classification_report(y_test, pred, labels=["negative","neutral","positive"], zero_division=0))
```

4. Compare class-wise precision, recall and F1

Accuracy alone can hide weak performance on a minority class. Precision indicates how often predictions for a class are correct; recall indicates how many true examples of that class were found; F1 balances precision and recall.

Python code cell

```
summary = []
```

```
for name, r in results.items():
    for label in ["negative","neutral","positive"]:
        m = r["report"][label]
        summary.append({"model": name, "class": label, "precision": m["precision"],
                        "recall": m["recall"], "f1-score": m["f1-score"],
                        "support": m["support"], "accuracy": r["accuracy"]})
metrics = pd.DataFrame(summary)
display(metrics.round(3))
```

Python code cell

```
best_name = max(results, key=lambda name: results[name]["report"]["macro avg"]["f1-score"])
best = results[best_name]
print("Best by macro-F1 on this split:", best_name)
ConfusionMatrixDisplay.from_predictions(
    y_test, best["pred"], labels=["negative","neutral","positive"],
    display_labels=["Negative","Neutral","Positive"], cmap="Blues", values_format="d"
)
plt.title(f"Confusion matrix — {best_name}")
plt.tight_layout()
plt.show()
```

5. Common words and phrases by class

These are descriptive top TF-IDF terms, not proof that a word always expresses one sentiment. For example, “not good” depends on phrase context and negation.

Python code cell

```
from sklearn.feature_extraction.text import TfidfVectorizer
for label in ["negative", "neutral", "positive"]:
    subset = df.loc[df["sentiment"] == label, "reviewText"]
    if len(subset) == 0:
        continue
    vec = TfidfVectorizer(ngram_range=(1,2), stop_words="english", max_features=15)
    mat = vec.fit_transform(subset)
    scores = np.asarray(mat.mean(axis=0)).ravel()
    terms = np.array(vec.get_feature_names_out())
    order = scores.argsort()[-10:][::-1]
    plt.figure(figsize=(7,3))
    plt.barh(terms[order][::-1], scores[order][::-1])
    plt.title(f"Common TF-IDF terms — {label}")
    plt.xlabel("Mean TF-IDF")
    plt.tight_layout()
    plt.show()
```

6. Error review: sarcasm, mixed opinions, short reviews and negation

Inspect examples where the model prediction differs from the mapped label. These labels come from ratings, so a mismatch can mean model error, rating-text disagreement, or a limitation of the mapping rule.

Python code cell

```
error_df = pd.DataFrame({
    "reviewText": X_test.reset_index(drop=True),
    "mapped_label": y_test.reset_index(drop=True),
    "predicted_label": pd.Series(best["pred"])
})
errors = error_df[error_df["mapped_label"] != error_df["predicted_label"]]
display(errors.head(30))
```

7. Review analyser prototype

The helper below predicts a label and shows class probabilities when the selected classifier supports them. Treat the output as an estimate, not a fact; the model can miss sarcasm, context, mixed opinions and domain-specific language.

Python code cell

```
def analyse_review(review_text, model=best["model"]):
    probabilities = {}
    if hasattr(model, "predict_proba"):
        probs = model.predict_proba([review_text])[0]
        probabilities = dict(zip(model.classes_, probs.round(3)))
    label = model.predict([review_text])[0]
    caution = ("Prediction is uncertain; review the text manually."
```

```
                if probabilities and max(probabilities.values()) < 0.60
                else "Automated estimate only; context and sarcasm may be missed.")
    return {"review": review_text, "predicted_sentiment": label,
            "class_probabilities": probabilities, "note": caution}

for example in [
    "Excellent quality and easy to use",
    "It is okay, not great and not terrible",
    "Terrible battery life and poor support"
]:
    print(analyse_review(example), "\n")
```

8. Conclusion template

After running on the chosen public dataset, report: (1) dataset title/version/licence and cleaning counts; (2) label mapping; (3) test-set precision, recall and F1 for each class; (4) macro-F1 comparison between both models; (5) examples of errors; (6) limitations and reproducibility details. Do not fill in performance claims before running on the final dataset.

Standalone review analyser script

```
"""Simple product-review sentiment analyser prototype.
Usage: python review_analyzer.py "The battery is excellent"
Optional dataset: set REVIEWS_CSV=reviews.csv (columns reviewText,rating).
Without it, uses the included tiny demo dataset; demo-trained predictions are for demonstration only.
"""
import os, sys
import pandas as pd
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

path = os.environ.get("REVIEWS_CSV", "demo_reviews.csv")
df = pd.read_csv(path).dropna(subset=["reviewText", "rating"]).copy()
df["rating"] = pd.to_numeric(df["rating"], errors="coerce")
df = df.dropna(subset=["rating"])
df = df[df["rating"].between(1, 5)]
df["sentiment"] = np.select(
    [df["rating"] <= 2, df["rating"] == 3, df["rating"] >= 4],
    ["negative", "neutral", "positive"], default=None)
df = df.drop_duplicates(subset=["reviewText"])
if df["sentiment"].value_counts().min() < 2:
    raise SystemExit("Dataset must contain at least two examples per sentiment class.")

model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=50000)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42))
])
model.fit(df["reviewText"].astype(str), df["sentiment"])
review = " ".join(sys.argv[1:]).strip()
if not review:
    review = input("Enter a product review: ").strip()
if not review:
    raise SystemExit("No review supplied.")
label = model.predict([review])[0]
probs = dict(zip(model.classes_, model.predict_proba([review])[0].round(3)))
print("Predicted sentiment:", label)
print("Class probabilities:", probs)
print("Caution: this is an automated estimate; sarcasm, negation, mixed opinions, and domain shift may
    cause errors.")
```

Sample dataset notes

The included demo_reviews.csv has 30 synthetic illustrative reviews with 1–5 ratings. It is supplied only to check that the workflow executes. It is not a public dataset and is too small for credible model-performance claims. Replace it with a suitably sized public dataset and document its licence.